A Private AI Assistant on a 6 GB Laptop GPU

Design, code audit, fixes and evaluation plan for a local retrieval-augmented agent (work in progress)

Davide Maccabelli
Independent project

October 2026 — technical report

Abstract

This report describes a personal AI assistant that I built to run entirely on my own hardware, with no cloud service and no running cost. My goal is a private assistant that remembers what I tell it, answers from my documents and code, and later executes commands for me. The project went through three phases. First, I ran open models on the CPU of a free-tier ARM cloud server. Then I built a local version with Phi-3-mini on my laptop's CPU and a vector memory in LanceDB; this version worked well, because the model is small. Finally, I moved to DeepSeek-R1-Distill-Qwen-7B, quantised to 4 bits and loaded onto my NVIDIA RTX 3050 Laptop GPU with 6 GB of memory. The core of that version is about 120 lines of Python. The second version ran, but slowly, and I paused development before measuring it systematically. For this reason I report a memory budget, a review of my code and an evaluation plan rather than benchmark results. The memory budget puts the configured system at about 5.5 GiB of the 6 GiB available, close enough to the limit for the Windows driver to move part of it into system memory. Reviewing the code, I found three silent failures. First, the 4-bit KV-cache setting was never applied, because the library ignores parameters it does not know. Second, the prompt used a chat format whose markers are not in the model's tokenizer. Third, the context budget was tight enough to truncate answers. None of these raised an error. I then wrote a third version that fixes them. It sets the KV cache with the correct parameters and checks them at start-up, uses the model's own chat template, gives reasoning and answer separate token budgets, and rebuilds the memory with a multilingual embedder, chunks, sources and a relevance threshold. It also logs every answer and comes with unit tests and an evaluation script. I have not yet run it on the GPU, so this report still contains no benchmark results. I close with the security requirements for the command-execution step and a pre-registered evaluation protocol.

1 Introduction

Cloud assistants are convenient, but everything I type into them leaves my computer. For personal notes, code and credentials, that is a real cost. Open-weight models quantised to 4 bits now fit on consumer hardware, so a private assistant is possible in principle. The constraint is memory: a laptop GPU with 6 GB has to hold the model, its working memory for the conversation and the software that runs it.

I wanted the assistant to do three things, in this order:

1. work as a private, general-purpose assistant that remembers facts from one session to the next;
2. read my documents and code and answer from them (retrieval-augmented generation, RAG; Lewis et al., 2020);
3. execute commands and code on my computer or my server, with safeguards.

This report covers what I built towards the first two goals and what the third would require. It contains: (i) a minimal, working local RAG design and its history; (ii) a memory and latency

budget for a 6 GB laptop GPU; (iii) a review of my code that finds failure modes which are easy to miss because nothing crashes; and (iv) a protocol for measuring the system before claiming anything about it.

2 Background

Retrieval-augmented generation. Instead of relying only on what a model learned in training, a RAG system turns the question into a vector, retrieves the most similar stored texts and places them in the prompt (Lewis et al., 2020). Sentence-embedding models such as Sentence-BERT (Reimers and Gurevych, 2019) and its distilled variants built on MiniLM (Wang et al., 2020) make this cheap enough to run on a CPU.

Small and reasoning models. Phi-3-mini (3.8B parameters) was designed to run locally on modest devices (Abdin et al., 2024). DeepSeek-R1 introduced open models that write out a long chain of reasoning between `<think>` tags before answering; its distilled versions transfer this behaviour to models of 1.5 to 70 billion parameters (DeepSeek-AI, 2025). Reasoning helps with hard problems but multiplies the number of generated tokens, which matters on slow hardware.

Quantisation and the KV cache. Storing model weights with about 4.5 bits per parameter (the GGUF `Q4_K_M` format of llama.cpp) reduces a 7B model to under 5 GB. During generation the model also keeps a key–value (KV) cache for every token of context. It can be quantised too: llama.cpp supports 8-bit and 4-bit caches, with negligible loss at 8 bits and a noticeable but usable loss at 4 bits (McLeod, 2024). Google's TurboQuant reports quality-neutral KV caches at 3.5 bits per channel (Zandieh et al., 2025), but it was not available in the engine I used.

Prompt repetition. Repeating the user's question in the prompt improves accuracy for non-reasoning models without lengthening the output (Leviathan et al., 2025). The claim is explicitly limited to non-reasoning models.

3 History of the project

4 Design

Figure 1 shows the second version. The embedding model (`all-MiniLM-L6-v2`, 384 dimensions) runs on the CPU, and I installed a CPU-only build of PyTorch, so all GPU memory is left to the language model. Memories are stored as text and vectors in an embedded LanceDB table on disk. For each question, the three nearest memories are inserted into the prompt, and the model streams its reasoning and answer token by token. LanceDB ranks by Euclidean distance by default; because this embedding model outputs unit-length vectors (I checked the stored ones), the ranking is the same as by cosine similarity.

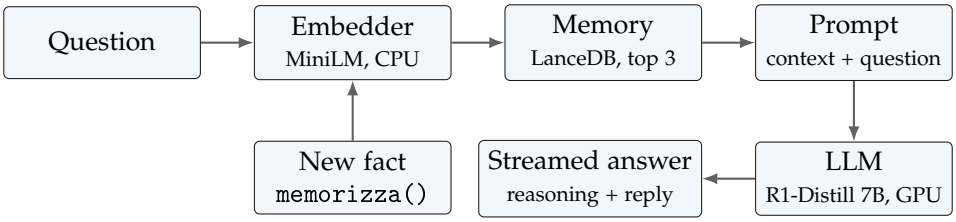


Figure 1: Architecture of the second version. Everything runs on my laptop.

In the first version I used Phi-3's own prompt format and a strict instruction: answer only from the retrieved context, and say so when the information is missing. To test it, I stored two long distractor passages on unrelated technical topics and one short "needle" with two numeric

Phase	Setup	What it added
0. Cloud server (late 2025)	Oracle Cloud Always Free ARM instance (4 cores, 24 GB RAM); plan for open models of 7–12B on CPU through Ollama, a command-executing assistant (Open Interpreter), Docker and layered backups	My first experience with self-hosted models and with giving an AI access to a system
1. Local CPU (31 March 2026)	Phi-3-mini-4k-instruct (`q4` GGUF, 2.39 GB) on my laptop's CPU; 2,048-token context; LanceDB memory; MiniLM embeddings	Persistent vector memory; a prompt that answers only from retrieved context; prompt repetition; a "needle in a haystack" stress test. It worked well
2. Local GPU (1 April 2026)	DeepSeek-R1-Distill-Qwen-7B (`Q4_K_M`, 4.68 GB) fully loaded onto my RTX 3050 Laptop GPU (6 GB) through a CUDA build of llama-cpp-python 0.3.19; 8,192-token context; streaming output	Reasoning model; larger context to read manuals and code; real-time output; code refactored into a class. It ran, but slowly
Paused (7 April 2026)	Project archived	—
3. Fixes (October 2026)	Same model and GPU; 4,096-token context with an 8-bit KV cache; multilingual memory	The fixes in Section 7, logging, unit tests and an evaluation script. Not yet run on the GPU

Table 1: Phases of the project. Dates come from file timestamps.

parameters, then asked for those parameters while telling the model to ignore the rest. This version worked well: at 3.8 billion parameters, Phi-3-mini is light enough for a laptop CPU.

In the second version I wanted better reasoning and a context large enough to read manuals and code, so I moved to a 7B reasoning model on the GPU. I loosened the instruction ("if data are missing, deduce them logically or state the uncertainty") to make use of the reasoning, and tested whether a stored credential-like string and an unrelated technical fact could be recalled together.

5 Memory and latency budget

Memory. The 7B model has 28 layers with 4 key–value heads of dimension 128 (grouped-query attention). Its KV cache therefore needs $2 \times 28 \times 4 \times 128 \times 2 = 57{,}344$ bytes per token at 16-bit precision: 0.44 GiB for 8,192 tokens, 0.23 GiB at 8 bits and 0.12 GiB at 4 bits. The weights take 4.36 GiB. On top of this, llama.cpp allocates compute buffers, and the CUDA context and Windows take their share; I estimate 0.6–0.9 GiB in total, a figure I still have to measure. Figure 2 compares the configurations.

As configured, the system needs about 5.5 GiB of the 6 GiB on the card. Any other program using the GPU (a browser with hardware acceleration, the display, another model) can push it over. Since driver 536.40, the Windows NVIDIA driver does not fail in that case. It moves the overflow to system memory, which is several times slower: reports put it at 5–10× (RunAIHome, 2026). A model that "runs, but slowly" is exactly the expected symptom.

Latency. Generating a token requires reading all the weights once. With 144 GB/s of memory bandwidth (Notebookcheck, 2023) and 4.68 GB of weights, the ceiling is about 30 tokens per second; real systems reach a fraction of it. A reasoning model then spends hundreds to thousands of tokens thinking before it answers. Figure 3 shows what this means in seconds. Even when everything fits on the GPU, a long reasoning trace takes a minute or more; with

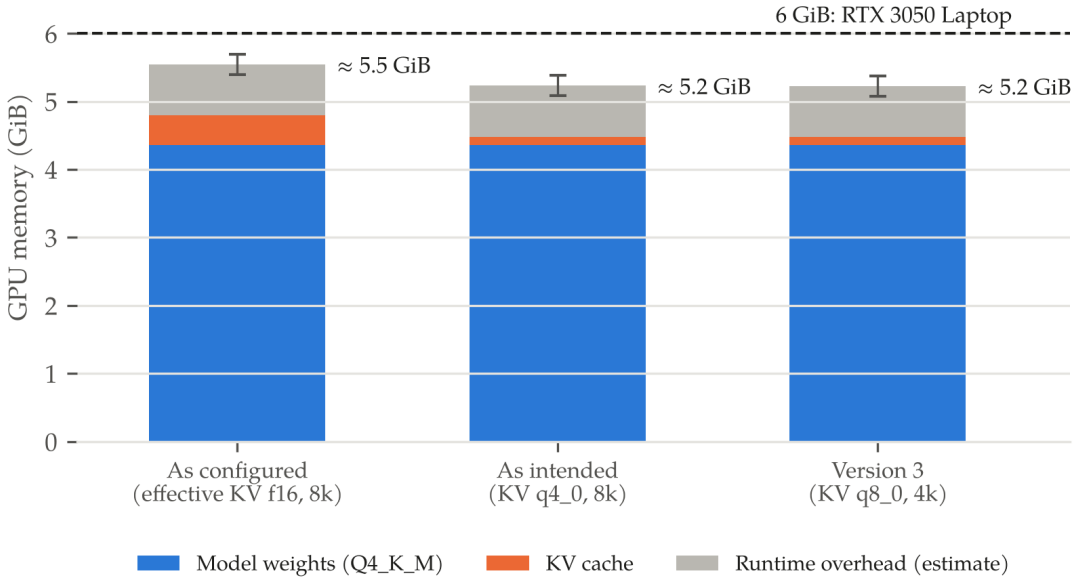


Figure 2: Estimated GPU memory for the 7B model. "As configured" is what actually ran, because the 4-bit cache setting was ignored (Section 6); "Version 3" is the new default. The grey part is an estimate.

memory spilling, several minutes. The setting `max_tokens=1500` also means that a long trace can be cut off before the final answer (version 3 gives reasoning and answer separate budgets; Section 7). The first version did not have this problem: Phi-3-mini answers directly, without a reasoning trace.

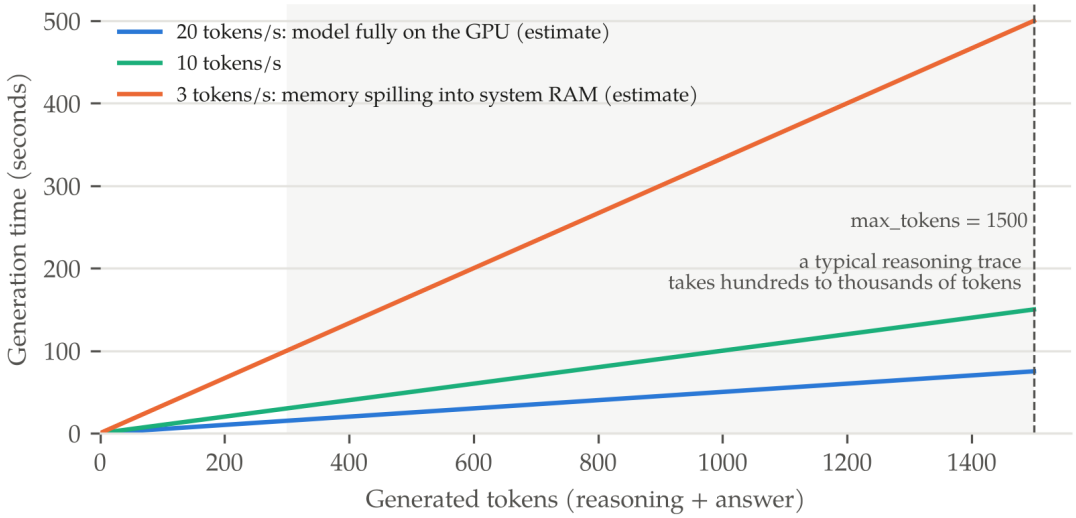


Figure 3: Generation time against the number of generated tokens at three speeds. The speeds are estimates that I will replace with measurements.

6 Code review

I reviewed the current source and reconstructed the first version from its compiled bytecode, because I had overwritten the source. Table 2 lists what I found. The common thread is that none of these problems produces an error: the program runs and prints an answer, so they only become visible by checking what actually happened.

Several choices were sound and I will keep them: the strict grounding instruction of version

#	Finding	Fix in version 3
F1	**KV-cache quantisation was never applied.** `kv_cache_type` is not a parameter of llama-cpp-python 0.3.19; the constructor accepts unknown keywords and drops them. The cache stayed at 16 bits in both versions (0.32 GiB more than intended at 8k).	I pass `type_k` and `type_v` with the GGML type constants (8 bits by default), keep flash attention on, and print at start-up the cache types that llama.cpp actually received.
F2	**Prompt format mismatch.** Version 2 wraps the prompt in ChatML markers (`<|im_start|>`), which do not exist in the R1-Distill tokenizer; its template uses `<|User|>` and `<|Assistant|>`, written with full-width bars. It also uses a system prompt and temperature 0.1, while DeepSeek advises no system prompt and a temperature of 0.5–0.7. Version 1 used Phi-3's format correctly.	I render the chat template stored in the GGUF file, put instructions and memories in the user turn, and use temperature 0.6 for reasoning models (0.3 otherwise).
F3	**Context budget.** Version 1 worked, but its stress test used most of the 2,048-token window (roughly 1,900–2,200 tokens by my estimate), leaving no margin for longer memories. In version 2, a reasoning trace can reach `max_tokens` before the answer starts.	I count the prompt tokens before generating, drop the least relevant memories until the prompt fits, and give reasoning its own budget. The reasoning is kept out of the stored answer.
F4	**Retrieval quality.** The embedder is trained on English, while my memories are in Italian; it reads only the first 256 word pieces; whole passages are stored as single memories; the top 3 are always inserted, however weak the match.	A multilingual embedder, chunks sized to what it can read, each with source and date, and a cosine-distance threshold.
F5	**Prompt repetition on a reasoning model.** The question is repeated twice, a technique shown to help non-reasoning models only (Leviathan et al., 2025). It suited Phi-3 better than R1.	Optional: on by default only for non-reasoning models, with a switch for the A/B test.
F6	**Reproducibility and privacy.** `requirements.txt` is empty; the stress test imports functions removed in version 2; nothing is logged; the project, including its memory database, lives in a cloud-synced folder.	Pinned dependencies, a rewritten stress test, 27 unit tests, a CSV log of every answer, and data in a local folder outside OneDrive.

Table 2: Findings of the code review and how version 3 addresses them.

1, a stress test that includes distractors and a misleading instruction, the CPU-only embedder that leaves the GPU to the language model, the CUDA build with flash attention and streaming output, and the gradual path from cloud CPU to local CPU to local GPU.

7 Version 3

Table 2 lists the fixes. Three of them needed design choices that are worth explaining.

Token budget. With a 4,096-token context I reserve 2,048 tokens for reasoning, 768 for the answer and 16 as a margin, which leaves 1,264 for the prompt. Before generating, I render the full prompt with the chat template and count its tokens; if it is too long, I drop the least relevant memories one at a time. During generation I count the reasoning tokens. If the model has not closed its `</think>` block when the reasoning budget runs out, I close it myself with a short sentence and let the model write the answer from where it stopped. This is the "budget forcing" idea of Muennighoff et al. (2025), used here to guarantee room for an answer rather than to make the model think longer. Because llama.cpp reuses the cached prefix, the second call does not recompute the prompt or the reasoning.

Memory. The new embedder, `paraphrase-multilingual-MiniLM-L12-v2`, has the same 384 dimensions as the old one but was trained on more than 50 languages (Reimers and Gurevych, 2020). It reads at most 128 word pieces, so I cut texts into chunks of about 120 tokens, along

sentence boundaries for prose and line boundaries for code, with a small overlap. Each chunk keeps its source and date, and the prompt cites memories by number. A memory enters the prompt only if its cosine distance from the question is below a threshold. For now I set it to 0.65; the evaluation script suggests a value by comparing the distances of the right memories with those of the nearest memory for questions that have no answer in memory. The data folder is now outside OneDrive.

Prompt. Retrieved text is labelled as data, and the instruction tells the model not to follow orders that appear inside it. This does not stop prompt injection, but it is a cheap first layer, and it will matter once the assistant can run commands.

Testing. 27 unit tests run in under a second without a GPU, using a fake model that emits one character per token and a bag-of-words embedder. They cover chunking, the threshold, the token budget, the separation of reasoning and answer (including a `</think>` tag split across two streamed pieces) and the forced close. They cannot cover the real model on the real GPU: that is the first thing I will check.

8 Requirements for the command-execution step

Letting the assistant run commands changes the threat model. The OWASP Top 10 for LLM applications lists prompt injection, excessive agency and weaknesses in vector stores among the main risks (OWASP, 2025). In this design, any retrieved document or memory becomes part of the prompt, so a sentence planted in a file could become an instruction. Rules written into the prompt ("never run `rm -rf`") are guidance for the model, not enforcement. Restrictions must live in the operating system, and some apparently limited permissions are not limited: for example, Docker's documentation warns that membership of the `docker` group grants root-level privileges (Docker, 2026). The minimum requirements for the next phase are:

- human confirmation for every command with side effects, showing the exact command;
- an explicit allowlist of commands and arguments, without wildcards;
- execution in a disposable container or virtual machine without access to the Docker socket or to secrets;
- retrieved text clearly separated from instructions, and never treated as an instruction;
- a log of every action, and backups taken before risky operations. The layered backup scheme I planned for my cloud server is a good basis.

9 Evaluation plan

I will write down the questions, metrics and pass thresholds before looking at any result, so that the choice of configuration cannot be bent to fit the data.

- **Data:** 50 questions with known answers, taken from my own notes and code, plus 10 whose answer is not in memory.
- **Retrieval:** recall@3, i.e. how often the passage that contains the answer is among the three retrieved.
- **Answers:** share of correct answers, share of answers that use only the retrieved context, and share of correct refusals when the information is missing.
- **Speed and memory:** time to first token, tokens per second, reasoning tokens per answer, and peak GPU memory from `nvidia-smi`.
- **Configurations:** models (Phi-3-mini, R1-Distill-Qwen-7B and a non-reasoning 7B instruct model), KV cache (16, 8, 4 bits), context (4k, 8k), prompt repetition on and off.

A configuration is preferred only if it is better on the pre-registered metric by more than the run-to-run noise measured on repeated runs. The script that comes with version 3 implements this plan. It loads the memories into a temporary store and reports recall@3 and a suggested threshold. It then runs each configuration three times with fixed seeds and records a fingerprint of the question file, so that I can show that the questions did not change after the first run.

10 Roadmap

1. **Done in version 3:** fixes F1–F6, logging, a rebuilt memory with sources and a threshold, and ingestion of text and code files from chosen folders.
2. Run version 3 on the GPU and check that the KV cache and the context are what I asked for.
3. Write the 60 evaluation questions, calibrate the threshold, run the evaluation plan and choose the model on the data. Phi-3-mini already worked well; a small non-reasoning model may be the right default for everyday questions, with the reasoning model kept for hard ones.
4. Read PDF and Word documents as well.
5. Add command execution only with the safeguards above.

11 Conclusion

A fully local assistant with long-term memory fits on a 6 GB laptop GPU, but only just: the margin is a few hundred megabytes, and when it is exceeded a reasoning model turns it into minutes of waiting. A small model such as Phi-3-mini gave me a usable assistant on the CPU alone. The most useful lesson I took from this project is that local AI stacks fail silently. A misspelled parameter, a chat template from another model family and a context that overflows by a few hundred tokens all leave the program running and answering. Measuring what actually happens is the step that makes the rest trustworthy. Version 3 fixes what I found and records every answer; measuring it is my next step.

Limitations. I did not run benchmarks or log outputs: version 1 worked well, version 2 ran but slowly, and that is all I can state. Version 3 passes its unit tests, but I have not yet run it with the real model on the GPU. The memory overheads and generation speeds in Figures 2 and 3 are estimates. The model files were no longer in the project folder when I reviewed it, so I could check their provenance only from the download metadata.

References

Abdin, M., et al. (2024). Phi-3 technical report: a highly capable language model locally on your phone. arXiv:2404.14219.

DeepSeek-AI (2025). DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948.

Docker Inc. (2026). Linux post-installation steps for Docker Engine: manage Docker as a non-root user. `https://docs.docker.com/engine/install/linux-postinstall/`.

Leviathan, Y., Kalman, M., Matias, Y. (2025). Prompt repetition improves non-reasoning LLMs. arXiv:2512.14982.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems* 33.

Muennighoff, N., Yang, Z., Shi, W., et al. (2025). s1: simple test-time scaling. arXiv:2501.19393.

McLeod, S. (2024). Bringing K/V context quantisation to Ollama. `https://smcleod.net/2024/12/bringing-k/v-context-quantisation-to-ollama/`.

Notebookcheck (2023). NVIDIA GeForce RTX 3050 6GB Laptop GPU: benchmarks and specs.

OWASP Foundation (2025). OWASP Top 10 for Large Language Model Applications, 2025 edition.

Reimers, N., Gurevych, I. (2019). Sentence-BERT: sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*.

Reimers, N., Gurevych, I. (2020). Making monolingual sentence embeddings multilingual using knowledge distillation. *Proceedings of EMNLP 2020*.

RunAIHome (2026). Shared GPU memory slow for local AI: the NVIDIA sysmem fallback fix. `https://runaihome.com/blog/shared-gpu-memory-slow-local-ai-sysmem-fallback-fix-2026/`.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M. (2020). MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems* 33.

Zandieh, A., Daliri, M., Hadian, M., Mirrokni, V. (2025). TurboQuant: online vector quantization with near-optimal distortion rate. arXiv:2504.19874.